— AUDITORÍA INTERNA · RONOCA DESARROLLOS, S.L.

# AUDITORÍA DEL CRM

Dinero, seguridad, técnica y mejoras del CRM DTF · DTI
Informe y plan de acción · 17 de octubre de 2026

**MAPA DEL DINERO ★ CRUCE DE CIFRAS ★ SEGURIDAD ★ TÉCNICA ★ MEJORAS DEL CRM ★ PLAN DE ACCIÓN**

**3**
acciones de seguridad urgentes, una tarde de trabajo

**10**
mejoras del CRM priorizadas por impacto en caja y margen

**1-1-2027**
fecha límite: Verifactu con proveedor certificado

## Resumen ejecutivo

**La base es buena.** Los importes se guardan congelados, las facturas no se pueden editar, cada cambio queda registrado con su autor y las cifras tienen una sola definición. En un CRM hecho a medida, eso es poco habitual.

**Lo que falla no es el cálculo, es el control:**

- se entra con solo una contraseña;
- quedan claves de las tiendas sin cifrar;
- las comprobaciones automáticas no prueban la versión real que se despliega (por eso se cayó producción el 16-10);
- no salta ninguna alerta cuando algo se rompe.

**Donde más dinero hay que ganar:**

- cobrar antes, con avisos automáticos y pago por enlace;
- conocer el margen real, con el coste por metro de verdad y la rentabilidad por cliente;
- reactivar a los clientes que han dejado de comprar.

Este informe sale de revisar el código y la base de datos del CRM. Las cifras de producción se obtienen con las dos consultas de solo lectura del Anexo; ejecútalas y el diagnóstico pasa de «cómo está montado» a «cómo están tus números».

# 1 · MAPA DEL DINERO

El dinero se apunta en seis sitios, que llamamos libros. Lo importante es cómo se conectan entre ellos.

| Libro | Qué apunta | Dónde vive |
|---|---|---|
| **Ventas DTF** | Pedidos de las tiendas WooCommerce, sus cobros, tickets y facturas | `pedidos`, `cobros`, `facturas` |
| **Ventas textil** | Lo mismo para la personalización de prendas | `textil_pedidos`, `cobros`, `textil_facturas` |
| **Compras y gastos** | Facturas recibidas (todas, no solo las del textil) y gastos fijos | `textil_compras`, `gerencia_gastos_fijos` |
| **Banco** | Extractos y su enlace con facturas y compras | `banco_movimientos`, `banco_conciliaciones` |
| **Caja** | Efectivo y aportaciones de los socios. Un cobro en efectivo crea solo su apunte. | `caja_movimientos` |
| **Inversión** | Lo invertido en la empresa | `inversion_movimientos` |

## El recorrido de una venta

1. Pedido (WooCommerce o a mano).
2. Cobro (web, tarjeta, transferencia o efectivo).
3. Ticket o factura, con número correlativo y sin huecos.
4. Movimiento en el extracto del banco.
5. Conciliación: verde si es seguro, ámbar si hay que revisarlo.
6. Gerencia: vendido, cobrado, pendiente, margen, beneficio, IVA.

Grupo A = lo documentado (de ahí salen IVA y Sociedades). Grupo B = pedidos aún sin documento y gastos sin justificante. El grupo lo decide el sistema, no se elige a mano.

## Puntos fuertes

- **Importes congelados** en cada línea y coste por metro guardado en cada pedido: el pasado no se reescribe.
- **Facturas inmutables** y numeración con bloqueo; solo se puede borrar la última de cada serie.
- **Registro de auditoría encadenado**: toda alta, cambio y baja con su autor.
- **Una sola definición por cifra** (unas 50, con su ⓘ): el Dashboard y Gerencia no pueden contradecirse.
- **Conciliación prudente**: verde y ámbar, un pago para varias facturas y traspasos entre cuentas propias.

## Puntos débiles

| Prioridad | Hallazgo | Por qué importa |
|---|---|---|
| Alta | Solo se puede casar con el banco lo que tiene **factura** (o una compra). Un pedido con ticket, o cobrado sin documento, no puede enlazarse con su ingreso. | Parte del dinero del banco se queda sin explicar, y no ves qué está cobrado y comprobado en el banco. |
| Media | Una venta en efectivo apuntada directamente en Caja, sin registrar el cobro en el pedido | Ese dinero no cuenta en «cobrado» y el pedido sigue como pendiente. |
| Media | Pedidos con estado único; los tres estados (pago, producción, envío) están a medias | Un pedido enviado y sin cobrar puede parecer cerrado. |

| | | |
|---|---|---|
| Media | Clientes DTF y textil en fichas separadas, y sin regla de un cliente por email | Duplicados: inflan los «clientes nuevos» y bajan la recurrencia. |
| Baja | El margen DTF usa un coste por metro fijo, no el gasto real | Si suben los consumibles y no se actualiza el ajuste, el margen sale inflado. |

# 2 · CRUCE DE CIFRAS

La consulta del **Anexo A** devuelve 15 controles de los últimos 12 meses. Es de **solo lectura**: no cambia nada. Así se lee:

| Línea | Control | Debería salir | Si no sale, qué hacer |
|---|---|---|---|
| 01–03 | Pedidos tienda · cobrado · pendiente | 01 = 02 + 03, aproximadamente | Buscar cobros asignados al pedido equivocado |
| 04 | Cobrado de más | 0 | Revisar cobros duplicados, pedido a pedido |
| 05 | Pedidos sin ticket ni factura | Lo más bajo posible | Emitir los documentos con «Facturar» en Pedidos. **Es el mayor riesgo fiscal.** |
| 06 | Documento distinto al pedido | 0 | Rectificativa si el documento está mal |
| 07 | Facturas sin cobrar hace más de 30 días | Bajo | Reclamar. Ver la mejora 1 (avisos de cobro). |
| 08–09 | Ingresos y cargos del banco sin conciliar | Bajo | Conciliar, o marcar como traspaso |
| 10 | Conciliaciones en ámbar | 0 | Confirmarlas o deshacerlas |
| 11 | Ingresos de caja sin cobro de pedido | Solo mostrador sin pedido | Pasarlos a cobro del pedido que corresponda |
| 12–13 | Textil: pedidos y pendiente | Lo mismo que 01 y 03 | Lo mismo que en tiendas |
| 14 | Clientes con el mismo email | 0 | Fusionar fichas. Ver la mejora 7 (cliente único). |
| 15 | Cobros con fecha anterior al pedido | 0 | Corregir la fecha del cobro |

### Cómo seguir

Ejecuta la consulta en Supabase → SQL Editor y pásale el resultado a Claude en el CRM. Con esos números se prioriza la limpieza: primero la línea 05 (riesgo fiscal), luego la 07 (dinero que te deben) y después las 08 y 09 (banco sin explicar).

# 3 · SEGURIDAD

## Lo que está bien

- **La clave maestra de Supabase solo está en el servidor**, y las 13 funciones que la usan comprueban antes que eres administrador.
- No hay claves ni contraseñas en el código.
- Las facturas están blindadas y el registro de cambios no se puede manipular.
- No hay entradas abiertas desde fuera: es el CRM quien pide los datos a WooCommerce.

## Riesgos

| # | Prioridad | Riesgo | Arreglo |
|---|---|---|---|
| 1 | Alta | **Sin doble factor.** Con la contraseña de un socio se accede a todo: facturas, banco y claves de las tiendas. | Activar el doble factor de Supabase (aplicación del móvil) y exigirlo al entrar. |
| 2 | Alta | **Claves de WooCommerce todavía en claro**, como copia de respaldo de las del almacén cifrado (Vault). | Regenerar las claves en cada tienda, guardarlas solo cifradas y vaciar las columnas viejas. |
| 3 | Media | **La librería `nodemailer` tiene un fallo grave**: puede mezclar las credenciales de varios servidores de correo, y tú tienes uno por tienda. | Actualizar la librería. |
| 4 | Media | **Faltan cabeceras de seguridad web** (la política de contenido (CSP), HTTPS obligatorio (HSTS) y la que impide meter el panel dentro de otra web (X-Frame-Options)). | Añadirlas en `src/server.ts`. |
| 5 | Media | **Hay 18 políticas de acceso «FOR ALL»** en el historial de migraciones; 12 se sustituyeron después. | Confirmar con el Anexo B que ninguna sigue activa en una tabla de dinero. |
| 6 | Baja | **No consta copia recuperable al minuto** (PITR). | Revisarlo en el plan de Supabase y activarlo. |
| 7 | Baja | **No se ve bloqueo tras intentos fallidos** de inicio de sesión desde la app. | Confirmarlo con el Anexo B y activar el límite de Supabase Auth. |
| 8 | Baja | **Tres administradores con permisos idénticos.** | Crear un rol limitado antes de dar acceso a un empleado. |

# 4 · TÉCNICA

### Por qué se cayó producción el 16-10 con todo en verde

- La integración continua compila en **modo desarrollo**.
- Compila para **otro proveedor** (Cloudflare), y producción está en Vercel.
- **Nunca arranca el servidor**, y el fallo (`__exportAll`) aparecía justo al arrancar.

Se arregla con un paso más en CI: compilar como producción, arrancar el servidor y pedir la página de inicio.

| # | Prioridad | Hallazgo | Arreglo |
|---|---|---|---|
| 1 | Alta | CI no prueba la versión real | Compilar como producción, arrancar el servidor y pedir la página de inicio |
| 2 | Alta | No hay alertas de errores en producción | Servicio de alertas (Sentry o similar) con aviso por correo |
| 3 | Media | Los tipos de la base están desfasados (25 de unas 60 tablas) y hay 275 sitios sin comprobar (`any`) | Regenerar los tipos y comprobar en CI que están al día |
| 4 | Media | Las funciones del servidor que mueven dinero no tienen pruebas | Probar 5 flujos: emitir, cobrar, conciliar, borrar la última factura y sincronizar Woo |
| 5 | Media | Ficheros de más de 1.300 líneas | Partirlos por tema cuando se toquen |
| 6 | Baja | Carga inicial de unos 740 KB de código | Cargar Gerencia, gráficos y PDF solo al abrirlos |
| 7 | Baja | `bun.lock` de Lovable, que no se puede usar fuera de Lovable | Borrarlo |

# 5 · MEJORAS DEL CRM QUE YO HARÍA

Ordenadas por lo que dejan en caja y en margen frente a lo que cuestan. Las cinco primeras aprovechan datos que **ya existen**: casi todo es pantalla y automatismo, no cambiar la base.

## 1 · Avisos de cobro automáticos

Hacer ya

Impacto: cobrar antes · Esfuerzo: medio · Base: correo por tienda y PDF de factura ya existen

**QUÉ HACE**

A los 7, 15 y 30 días de una factura sin pagar, el CRM envía un correo amable con el PDF y el importe. Se puede pausar para un cliente concreto y cada envío queda en la ficha.

**PASOS**

1. Plantilla de correo de recordatorio por tienda.
2. Tarea diaria que busca facturas pendientes.
3. Registro de avisos enviados y botón para pausar.
4. Indicador en Gerencia: «avisos enviados y cobrados después».

## 2 · Ficha de deuda y riesgo por cliente

Hacer ya

Impacto: no servir a morosos · Esfuerzo: bajo

**QUÉ HACE**

En cada cliente se ve lo que debe, desde cuándo, cuántos días tarda de media en pagar y una etiqueta: puntual, lento o moroso. Al crear un pedido, avisa si el cliente debe dinero.

**PASOS**

1. Cálculo en el dominio: deuda y días medios de pago.
2. Bloque en la ficha del cliente.
3. Aviso al crear un pedido o presupuesto.

## 3 · Rentabilidad por cliente y por tienda

Hacer ya

Impacto: saber a quién cuidar y a quién subir el precio · Esfuerzo: bajo (los datos ya están)

**QUÉ HACE**

Muestra un ranking de clientes por margen, no por ventas, con el margen por metro de cada uno. Los clientes que compran mucho y te dejan poco salen en rojo.

**PASOS**

1. Agrupar el margen que ya calcula Gerencia por cliente.
2. Pestaña «Rentabilidad» en Clientes.
3. Filtro por tienda y por periodo.

## 4 · Precio mínimo por metro

Hacer ya

Impacto: no vender por debajo del margen · Esfuerzo: bajo

**QUÉ HACE**

En Ajustes fijas el margen mínimo. Si un presupuesto, pedido o descuento lo rompe, sale un aviso en ámbar antes de guardar.

**PASOS**

1. Ajuste «margen mínimo por metro».
2. Comprobación en presupuesto y pedido.
3. Lista de pedidos por debajo del mínimo en Gerencia.

## 5 · Clientes dormidos, con acción

Este trimestre

Impacto: ventas recuperadas casi gratis · Esfuerzo: bajo (Gerencia ya los cuenta)

**QUÉ HACE**

Lista los clientes que compraban con regularidad y llevan más tiempo del habitual sin pedir. Botón para enviarles un correo o una oferta y registro de quién vuelve a comprar.

**PASOS**

1. Lista a partir de la cifra «dormidos» de Gerencia.
2. Plantilla de reactivación.
3. Seguimiento: cuántos vuelven en 30 días.

## 6 · Coste por metro real

Este trimestre

Impacto: el margen deja de ser una estimación · Esfuerzo: medio

**QUÉ HACE**

Las compras de tinta, film y polvo se etiquetan como consumibles DTF. Cada mes se divide su coste entre los metros impresos y se compara con el coste configurado. Avisa si se desvía más de un 10 %.

**PASOS**

1. Categoría «consumible DTF» en compras (ya hay categorías).
2. Cálculo mensual del coste real por metro.
3. Gráfico real frente a configurado en Gerencia › Margen.

## 7 · Previsión de caja a 30, 60 y 90 días

Este trimestre

Impacto: ver un mes flojo de tesorería con tiempo · Esfuerzo: medio

Saldo del banco + cobros pendientes (ponderados por cómo paga cada cliente) − gastos fijos − compras pendientes de pago. El resultado es una línea por semana con alertas si baja de un mínimo.

## 8 · Envío a la gestoría con un clic

Este trimestre

Impacto: menos horas de gestoría y menos errores · Esfuerzo: bajo (el ZIP de PDF ya existe)

Un ZIP por trimestre con las facturas emitidas y recibidas, el libro de IVA en Excel, el resumen de conciliación y la lista de lo que queda sin documentar.

## 9 · Pago por enlace en presupuestos y facturas

Más adelante

Impacto: cobro inmediato y conciliación automática · Esfuerzo: medio-alto (pasarela de pago)

Botón «Pagar» en el PDF y en el correo, con Bizum o tarjeta. El cobro entra solo en el pedido. En el textil sirve para pedir un anticipo al aceptar el presupuesto.

## 10 · Cliente único y cola de producción

Más adelante

Impacto: visión completa del cliente y nada se retrasa · Esfuerzo: alto

Fusionar las fichas de DTF y textil en un cliente por email, y terminar los tres estados de pedido. Con eso se hace una cola de producción: qué imprimir hoy, por fecha de entrega y en metros.

# 6 · PLAN DE ACCIÓN PASO A PASO

## Semana 1 · Seguridad y estabilidad

- ☐ **Doble factor:** en Supabase → Authentication → activar el doble factor (MFA, por aplicación del móvil); cada socio lo configura en su cuenta; luego exigirlo en el panel.
- ☐ **nodemailer:** actualizar la dependencia, pasar `npm run verify` y desplegar.
- ☐ **Claves WooCommerce:** en cada tienda, WooCommerce → Ajustes → Avanzado → API REST → revocar la clave y crear una nueva; pegarla en el CRM (se guarda cifrada); vaciar las columnas en claro con una migración nueva.
- ☐ **CI de producción:** añadir a `verificar.yml` la compilación de producción, el arranque del servidor y la petición a «/».
- ☐ **Alertas:** crear una cuenta en un servicio de alertas, conectarlo al servidor y al navegador, y recibir un correo con cada error nuevo.

## Semanas 2–4 · Que los números cuadren

- ☐ Ejecutar los Anexos A y B y revisar el resultado.
- ☐ Emitir los documentos de los pedidos del grupo B (línea 05).
- ☐ Conciliar el banco pendiente y confirmar los enlaces en ámbar (líneas 08–10).
- ☐ Fusionar los clientes duplicados (línea 14).
- ☐ Regenerar los tipos de Supabase y añadir la comprobación a CI.
- ☐ Probar los 5 flujos de dinero.
- ☐ Cabeceras de seguridad y PITR.

## Mes 2 · Cobrar antes y proteger el margen

- ☐ Mejora 1: avisos de cobro automáticos.
- ☐ Mejora 2: ficha de deuda y riesgo por cliente.
- ☐ Mejora 3: rentabilidad por cliente y tienda.
- ☐ Mejora 4: precio mínimo por metro.

## Trimestre · Gestionar con datos reales

- ☐ Mejora 5: clientes dormidos.
- ☐ Mejora 6: coste por metro real.
- ☐ Mejora 7: previsión de caja.
- ☐ Mejora 8: paquete trimestral para la gestoría.

### Antes del 1 de enero de 2027 · Verifactu (obligatorio)

- ☐ Decidir si el CRM pasa a emitir las facturas que se declaran o si sigue la gestoría.
- ☐ Si las emite el CRM: elegir un **proveedor certificado** que se encargue de la huella, el encadenamiento y el envío a la AEAT, e integrarlo por su API.
- ☐ Revisar la regla de «borrar la última factura»: una factura enviada a la AEAT no se puede borrar.
- ☐ Revisar la fecha de los documentos: hoy se usa la fecha del pedido.

### Más adelante

- ☐ Mejora 9: pago por enlace.
- ☐ Banco conectado automáticamente (PSD2).
- ☐ Mejora 10: cliente único, tres estados de pedido y cola de producción.
- ☐ Partir los ficheros grandes y reducir la carga inicial.

# A · CONSULTA DE CIFRAS (SOLO LECTURA)

Supabase → SQL Editor → pegar → Run. No modifica nada.

```
WITH
p AS (SELECT id, total FROM public.pedidos
      WHERE fecha_pedido >= now() - interval '12 months' AND estado::text <> 'cancelado'),
cob AS (SELECT pedido_id, sum(importe) AS cobrado FROM public.cobros
        WHERE pedido_id IS NOT NULL GROUP BY pedido_id),
dev AS (SELECT pedido_id, sum(importe) AS devuelto FROM public.pedido_devoluciones GROUP BY pedido_id),
pc AS (SELECT p.id, p.total - coalesce(dev.devuelto,0) AS neto, coalesce(cob.cobrado,0) AS cobrado
       FROM p LEFT JOIN cob ON cob.pedido_id = p.id LEFT JOIN dev ON dev.pedido_id = p.id),
fv AS (SELECT * FROM public.facturas WHERE estado::text NOT IN ('borrador','anulada')),
tp AS (SELECT id, total FROM public.textil_pedidos
       WHERE fecha >= current_date - 365 AND estado <> 'cancelado'),
tcob AS (SELECT textil_pedido_id, sum(importe) AS cobrado FROM public.cobros
         WHERE textil_pedido_id IS NOT NULL GROUP BY textil_pedido_id)
SELECT '01 Pedidos tienda (12 meses), neto' AS control, count(*) AS cantidad, round(sum(neto),2) AS importe
FROM pc
UNION ALL SELECT '02 Cobrado de esos pedidos', count(*) FILTER (WHERE cobrado>0), round(sum(cobrado),2)
FROM pc
UNION ALL SELECT '03 Pendiente de cobro', count(*), round(sum(neto-cobrado),2) FROM pc WHERE neto-cobrado >
0.01
UNION ALL SELECT '04 Cobrado DE MÁS', count(*), round(sum(cobrado-neto),2) FROM pc WHERE cobrado-neto >
0.01
UNION ALL SELECT '05 Pedidos SIN ticket ni factura', count(*), round(sum(neto),2)
  FROM pc WHERE NOT EXISTS (SELECT 1 FROM fv WHERE fv.pedido_id = pc.id)
UNION ALL SELECT '06 Documento distinto al pedido', count(*), round(sum(fv.total-pc.neto),2)
  FROM fv JOIN pc ON pc.id = fv.pedido_id WHERE abs(fv.total-pc.neto) > 0.01
UNION ALL SELECT '07 Facturas sin cobrar +30 días', count(*), round(sum(total),2)
  FROM fv WHERE estado::text = 'emitida' AND fecha < current_date - 30
UNION ALL SELECT '08 Ingresos banco sin conciliar', count(*), round(sum(importe),2)
  FROM public.banco_movimientos m WHERE importe > 0 AND traspaso_con IS NULL
   AND NOT EXISTS (SELECT 1 FROM public.banco_conciliaciones bc WHERE bc.movimiento_id = m.id)
UNION ALL SELECT '09 Cargos banco sin conciliar', count(*), round(sum(importe),2)
  FROM public.banco_movimientos m WHERE importe < 0 AND traspaso_con IS NULL
   AND NOT EXISTS (SELECT 1 FROM public.banco_conciliaciones bc WHERE bc.movimiento_id = m.id)
UNION ALL SELECT '10 Conciliaciones en ámbar', count(DISTINCT grupo), round(sum(importe),2)
  FROM public.banco_conciliaciones WHERE estado = 'revisar'
UNION ALL SELECT '11 Ingresos de caja sin cobro', count(*), round(sum(importe),2)
  FROM public.caja_movimientos cm WHERE categoria::text = 'ingreso'
   AND NOT EXISTS (SELECT 1 FROM public.cobros c WHERE c.caja_movimiento_id = cm.id)
UNION ALL SELECT '12 Pedidos textil (12 meses)', count(*), round(sum(total),2) FROM tp
UNION ALL SELECT '13 Pendiente de cobro textil', count(*), round(sum(tp.total-coalesce(tcob.cobrado,0)),2)
  FROM tp LEFT JOIN tcob ON tcob.textil_pedido_id = tp.id WHERE tp.total-coalesce(tcob.cobrado,0) > 0.01
UNION ALL SELECT '14 Clientes repetidos (email)', count(*), sum(n-1)
  FROM (SELECT lower(trim(email)) e, count(*) n FROM public.clientes
        WHERE coalesce(trim(email),'') <> '' GROUP BY 1 HAVING count(*) > 1) d
UNION ALL SELECT '15 Cobros anteriores al pedido', count(*), round(sum(c.importe),2)
  FROM public.cobros c JOIN public.pedidos pe ON pe.id = c.pedido_id WHERE c.fecha < pe.fecha_pedido::date;
```

# B · CONSULTA DE SEGURIDAD (SOLO LECTURA)

```
SELECT 'Tabla SIN RLS' AS tipo, c.relname AS objeto, '' AS detalle
  FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
 WHERE n.nspname = 'public' AND c.relkind = 'r' AND NOT c.relrowsecurity
UNION ALL SELECT 'Política FOR ALL', tablename, policyname FROM pg_policies
 WHERE schemaname = 'public' AND cmd = 'ALL'
UNION ALL SELECT 'Política abierta (true)', tablename, policyname || ' · ' || cmd FROM pg_policies
 WHERE schemaname = 'public' AND (qual = 'true' OR with_check = 'true')
UNION ALL SELECT 'Función privilegiada sin search_path', p.proname, ''
  FROM pg_proc p JOIN pg_namespace n ON n.oid = p.pronamespace
```

```
 WHERE n.nspname = 'public' AND p.prosecdef
   AND NOT coalesce(array_to_string(p.proconfig, ',') LIKE '%search_path%', false)
UNION ALL SELECT 'Bucket público', id, '' FROM storage.buckets WHERE public
UNION ALL SELECT 'Claves Woo en claro', count(*)::text, 'tiendas con consumer_secret'
  FROM public.tienda_credenciales WHERE coalesce(consumer_secret,'') <> ''
UNION ALL SELECT 'Usuario SIN 2FA', u.email, '' FROM auth.users u
 WHERE NOT EXISTS (SELECT 1 FROM auth.mfa_factors f WHERE f.user_id = u.id AND f.status = 'verified')
ORDER BY 1, 2;
```

El bucket `logos` es público a propósito (los logos salen en los PDF). Si la tabla `tienda_credenciales` no existe con ese nombre, quita esa línea.